

Data Permissions, Sources, Tools & Connections

Enterprise Data Survival

Professional Development

TOAD • Teradata • Vertica • Hive • ODBC • Oracle • PySpark • SAS

What this course covers

- **Permissions** — prod vs dev vs uat, confidential levels
- **Sources** — where data lives, how to find it
- **Tools** — TOAD, Teradata, PySpark, PowerBroker
- **Connections** — ODBC, drivers, error handling
- **Queries** — SQL optimization, conversion to PySpark
- **SAS** — logistic regression, data cleaning

Enterprise data reality

Different data sizes

- **Data** — Excel, CSV, small tables
- **Big Data** — Terabytes, Hadoop, Hive
- **Very Big Data** — Petabytes, distributed clusters

Different formats & locations

- Teradata, Vertica, Oracle, Hive, SQL Server
- On-premise vs cloud vs hybrid
- Prod / Dev / UAT / Sandbox environments

You won't know where anything is. That's normal.

Permissions hell

Typical permission levels

- **Prod** — live data, read-only usually
- **Dev** — test data, can write
- **UAT** — user acceptance testing
- **Sandbox** — play around

The problem

- You have Dev access but need Prod → error
- You have Prod access but not UAT → can't test
- You have no access at all → raise a ticket (wait 2 weeks)

You will see "Table not found" when table exists. It's permissions.

Confidential data levels

Confidential

- Internal reports
- Non-public metrics
- Normal approval needed

Very Confidential

- PII (personally identifiable info)
- Customer data
- Legal/compliance review required

What this means for you

- Different database schemas
- Different ODBC connections
- Extra approval forms
- "You are not authorized" errors

How to raise a permission request

- 1 Get a new error (e.g., "Table not found")
- 2 Search wiki for "permission" + database name
- 3 Find the request form (SharePoint, ServiceNow, Jira)
- 4 Fill it out (database, schema, table, read/write)
- 5 Wait 2 days to 2 weeks
- 6 Chase via email, Teams, Slack
- 7 Eventually get access (or not)

No one will tell you this process. You find it yourself.

Common permission errors

Error message	What it actually means
"Table not found"	Table exists, you can't see it
"Database not found"	Wrong environment or no access
"Access denied"	You need approval
"ODBC connection failed"	Wrong driver or no network permission
"User not authorized"	Ask your manager (who won't answer)

Connection types in enterprise

ODBC (Open Database Connectivity)

- **Manual ODBC** — you configure driver, DSN, server
- **Automatic ODBC** — pre-configured, just use it

Other connection methods

- JDBC (Java)
- Native drivers (Teradata, Oracle)
- REST APIs
- SSH tunnels

TOAD connection screens

TOAD Data Point (TDP)

- Connect to Teradata, Oracle, SQL Server, Hadoop
- Need: Server name, port, database, username, password
- Authentication: LDAP, Windows, or database auth

Common TOAD issues

- "Unable to connect" → wrong driver or firewall
- "Invalid credentials" → password expired
- "Timeout" → network slow or query too big

Teradata connections

Via ODBC driver

- Teradata ODBC driver installed (version matters)
- DSN configured with server IP and database
- Connection string: `DBCName=server; Database=schema;`

Common Teradata errors

- "[Teradata] Driver] No server found" → wrong host
- "Logon failed" → wrong user/pass or expired
- "Query timeout" → query runs too long

Other databases — Vertica, Hive, Oracle

Vertica

- Columnar storage
- ODBC/JDBC
- High performance

Hive

- Hadoop layer
- SQL-like (HiveQL)
- Slow but big data

Oracle

- Traditional RDBMS
- TNS names file
- Enterprise standard

Each has its own permission system. Each has its own error messages.

Connection errors you will see

- "Unable to connect to server" — network down or firewall
- "Driver not found" — install ODBC driver
- "Authentication failed" — wrong password or expired
- "TNS could not resolve" — Oracle TNS misconfigured
- "SSL handshake failed" — certificate issue
- "Connection refused" — server not accepting
- "Timeout expired" — slow network or query

You will see all of these. Probably in one day.

Enterprise data tools

Tool	Use
Teradata	Data warehouse, big SQL
TOAD	Query editor, multiple DB connections
PySpark	Big data processing, Python on clusters
PowerBroker	Admin account escalation
SAS	Statistics, logistic regression

PySpark on clusters

Setup

- Connect to master node (SSH)
- Submit job to cluster
- Data distributed across worker nodes

Common PySpark issues

- "Master not found" — wrong IP
- "Out of memory" — executor too small
- "Stage failed" — data skew or missing partition
- "Permission denied" — wrong Linux user

Enterprise queries are not simple

What you will see

- 200+ line SQL queries
- 15+ joins across 10 tables
- Nested subqueries, CTEs, window functions
- No comments, no documentation

How to survive

- Run small `SELECT COUNT(*)` first
- Add `LIMIT 10` or `SAMPLE`
- Comment as you go
- Break into smaller CTEs

Joins you will use

Join type	When to use
INNER JOIN	Only matching rows
LEFT JOIN	All rows from left table
RIGHT JOIN	All rows from right table
FULL OUTER JOIN	All rows from both
CROSS JOIN	Cartesian product (dangerous)

Mismatched keys, duplicate rows, and NULLs will break everything.

Using Copilot/AI for queries

What AI can do

- Convert SQL to PySpark
- Explain long queries
- Suggest optimizations
- Write simple queries from scratch

What AI will do wrong

- Hallucinate columns that don't exist
- Ignore your database dialect
- Create inefficient joins
- Give confident wrong answers

Never run AI-generated code without checking.

Making queries faster

- 1 Use WHERE to filter early
- 2 Avoid SELECT * — name columns
- 3 Use EXPLAIN to see the plan
- 4 Add indexes on join columns
- 5 Partition large tables
- 6 Avoid functions in WHERE clauses
- 7 Use UNION ALL instead of UNION if possible

Slow queries will be killed by the DBA. You will get angry emails.

SAS in enterprise

What SAS is used for

- Logistic regression (credit risk, fraud)
- Data cleaning and transformation
- Statistical analysis
- Reporting (old school)

SAS code structure

- DATA step — manipulate data
- PROC step — run procedures
- PROC LOGISTIC — regression
- PROC FREQ — frequency tables

How to read SAS code (when you don't know SAS)

Example: Logistic regression

```
proc logistic data=mydata;  
  class gender region;  
  model default(event='1') = age income debt / selection=stepwise;  
run;
```

What it means

- `proc logistic` — run logistic regression
- `class` — categorical variables
- `model` — dependent variable = independent vars
- `event='1'` — predict default = 1

Common SAS data steps

```
data clean;  
  set raw;  
  if age = . then age = 0;  
  if income < 0 then delete;  
  format date yymmdd10.;  
run;
```

- Handle missing values (.)
- Remove negative or invalid values
- Format dates properly

How implementation team works

What they do

- Take model code from dev team
- Deploy to production environment
- Handle high resource requirements
- Monitor performance

Your interaction with them

- Raise Jira tickets
- Wait (sometimes weeks)
- Get "deployment failed" errors
- Chase via Teams/Slack

Step-by-step: running your first query

- 1 Get connection details (wiki, email, teammate)
- 2 Install ODBC driver (may need admin)
- 3 Configure DSN (server, database)
- 4 Open TOAD / Teradata / PySpark
- 5 Write a simple `SELECT COUNT(*) FROM table`
- 6 Run it
- 7 Get an error
- 8 Fix error (permissions, driver, typo)
- 9 Repeat until it works

Step 7 is guaranteed. Step 8 is where you learn.

Enterprise data survival cheat sheet

Problem	Solution
"Table not found"	Check permissions, then environment
"Connection failed"	Check ODBC driver, firewall, VPN
Query too slow	Add filters, limit, check indexes
SAS won't run	Check library path, permissions
No access to anything	Raise ticket, document, wait
AI gave wrong code	Test on small sample first

You will survive data

Every senior data person started exactly where you are now.
No permissions. Broken connections. Unreadable queries. Silent teams.
The difference?
They kept trying. They documented what worked. They asked better questions.

Now you go do the same.

Good luck. You will need it.